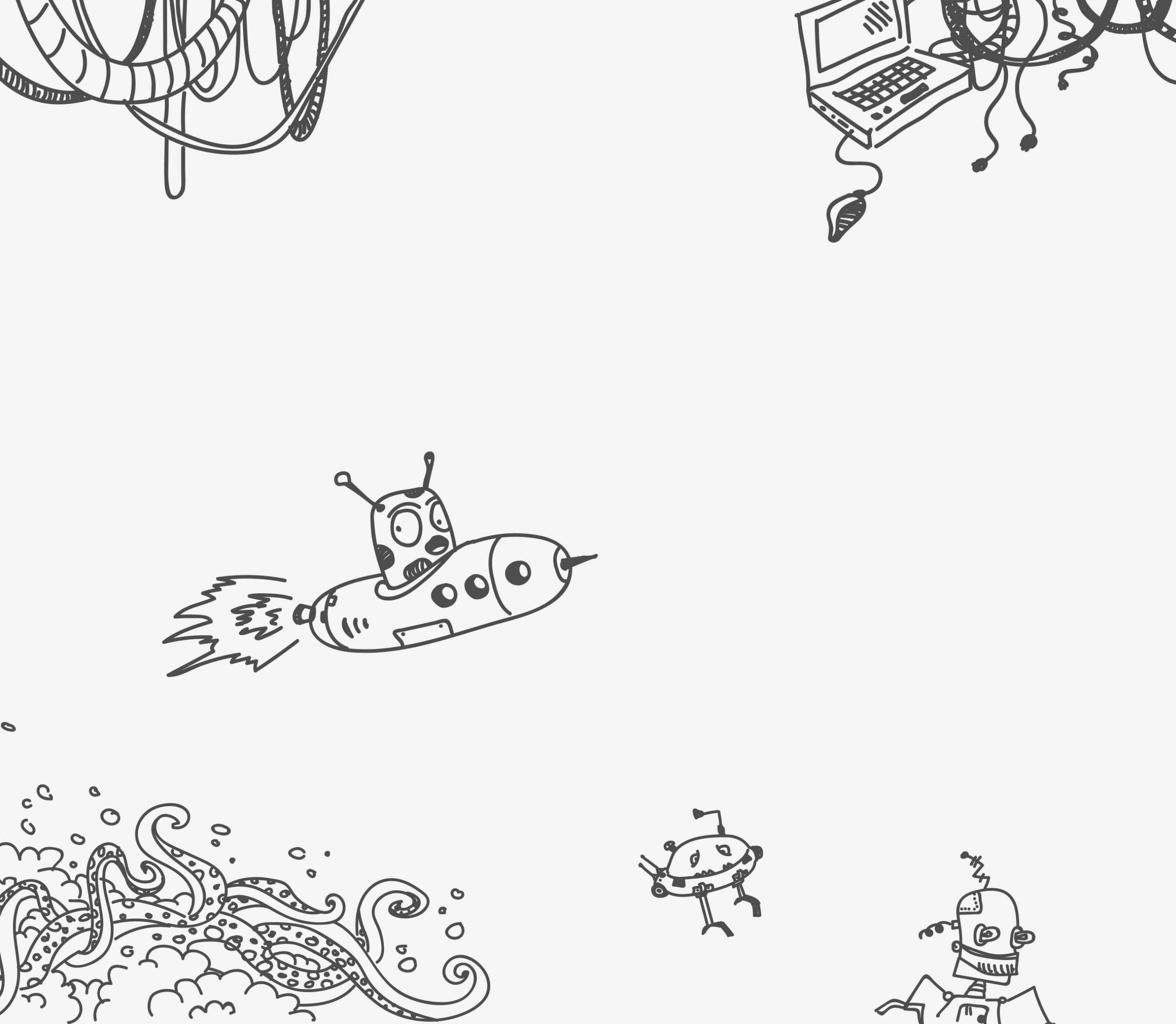


Decision tree in regression - an implementation example

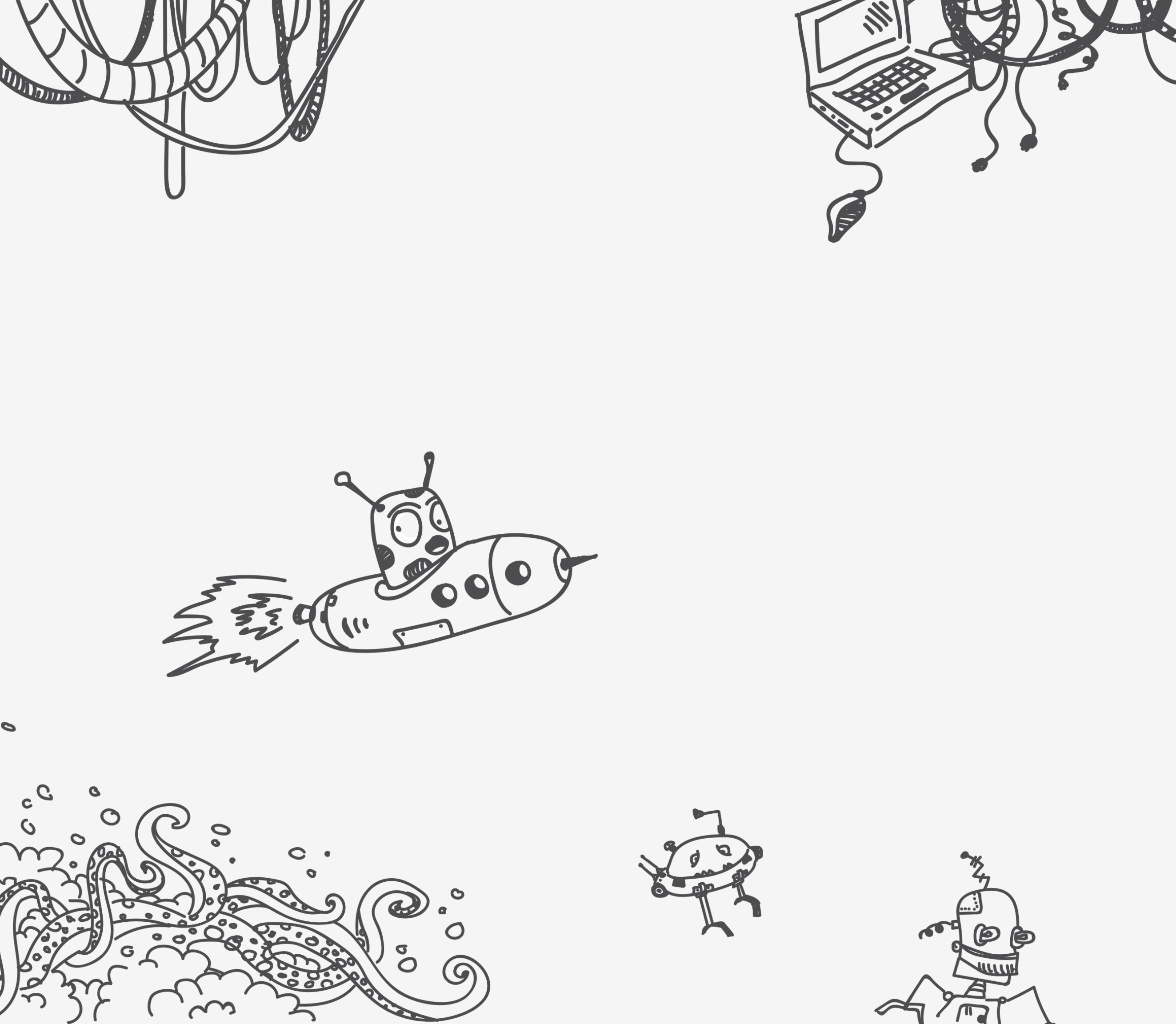
1



Introduction

Introduction

Previously, we learned how the decision tree algorithm works in the context of regression. Now that we have the theoretical basis, we can move on to discuss the implementation of the model using tools in Python. As in previous cases, the **Scikit-learn** library will prove to be an indispensable tool.



Generating the data

Generating the data

In order to best understand how the decision tree works by example, we will start with simple data, consisting of one independent variable (X) and the dependent variable we will predict. To create a dataset for the task, we can use the *make_regression* method from the **dataset** module. We will create a set consisting of 150 observations. We will also make some modification by squaring the dependent variable generated by the *make_regression* method. With this, we will show one of the advantages of the decision tree over previous algorithms.

```
from sklearn.datasets import make_regression

# Create synthetic regression dataset with 150 samples.
X, y = make_regression(n_samples=150, n_features=1, noise=30, random_state=42)

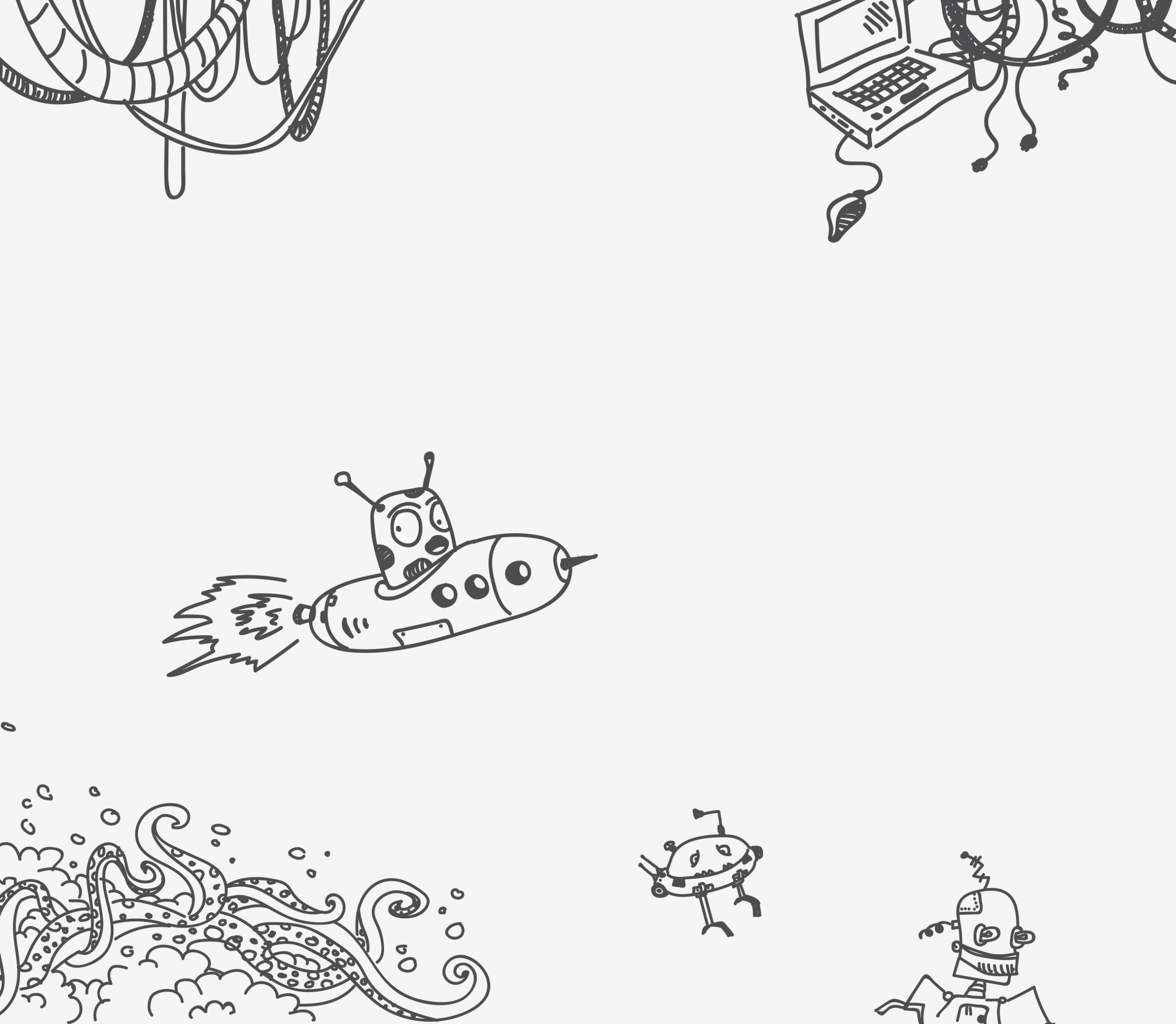
# Target variable is squared to make sure that relationship between X and y is not linear
y = y ** 2
```

Generating the data

Now let's look at the distribution of values in our dataset. We will use the **Matplotlib** library and the `.scatter()` method for this:

```
import matplotlib.pyplot as plt  
plt.scatter(X, y, s=10, color="r")
```





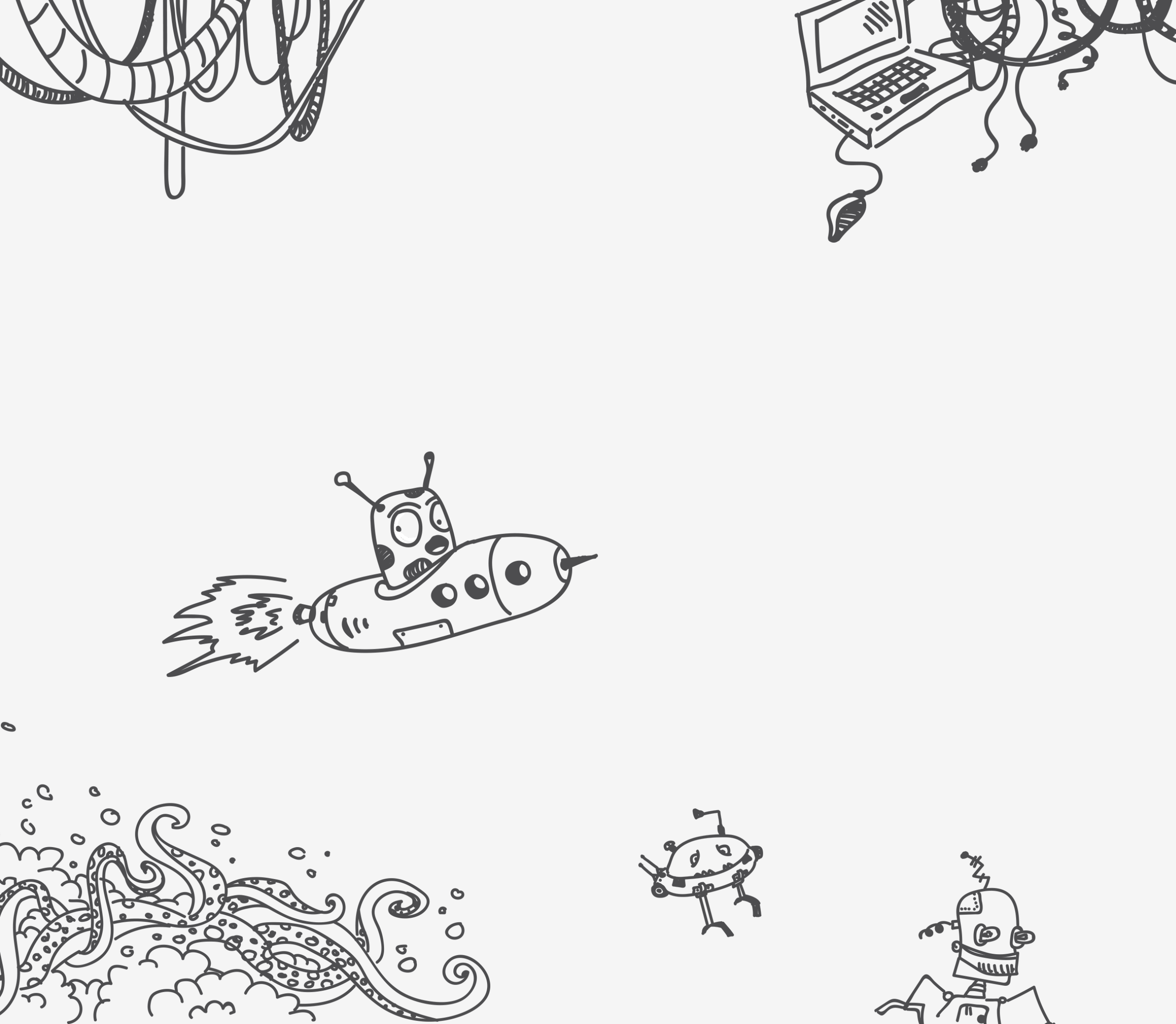
Generating the data

Generating the data

Looking at the data, we can observe a certain relationship between the **X** and **y** variables - the data points are arranged in a shape resembling a parabola. 2nd degree polynomial regression could give pretty good results on a dataset with such characteristics. What is most important is that the above relationship is definitely not linear. Our goal is to demonstrate that the decision tree can handle such data. Now that we have the data and know that we don't need to particularly focus on preparing it - standardizing or normalizing it - let's get to the point. We will create an instance of the decision tree model for the regression model. We import the appropriate class from the tree module and initialize the model:

```
from sklearn.tree import DecisionTreeRegressor

# Initialize decision tree model
reg_tree = DecisionTreeRegressor()
```



Building the model

Building the model

We have the model created. The next step will be to train the algorithm on the data we have:

```
# Train model  
reg_tree.fit(X, y)
```

We have trained our algorithm. In the course of the first presentation about the decision tree, one point that distinguishes models based on this algorithm surely caught your eye. That is the ease with which such a model can make decisions. We will now show how we can test this feature of the model - for simple data, such as the one we are working on, there are two methods.

The first involves generating a scatter plot with the values of the independent variable and the dependent variable plotted, and then generating a prediction on the same plane.

Building the model

The first involves generating a scatter plot with the values of the independent variable and the dependent variable plotted, and then generating a prediction on the same plane.

Using the **Numpy** library, let's generate an array with values from -3 to 3 (based on the X values that appeared in our dataset). We will convert the array to a single-column matrix and use it to make predictions through our model.

```
import numpy as np

# Plot dataset
plt.scatter(X, y, s=10, color="r")

# Plot model decision boundaries
plot_data = np.arange(-3, 3, 0.01).reshape(-1, 1)
plt.plot(plot_data, reg_tree.predict(plot_data), color="lightblue")
```

```
[<matplotlib.lines.Line2D at 0x7f4e0358bee0>]
```

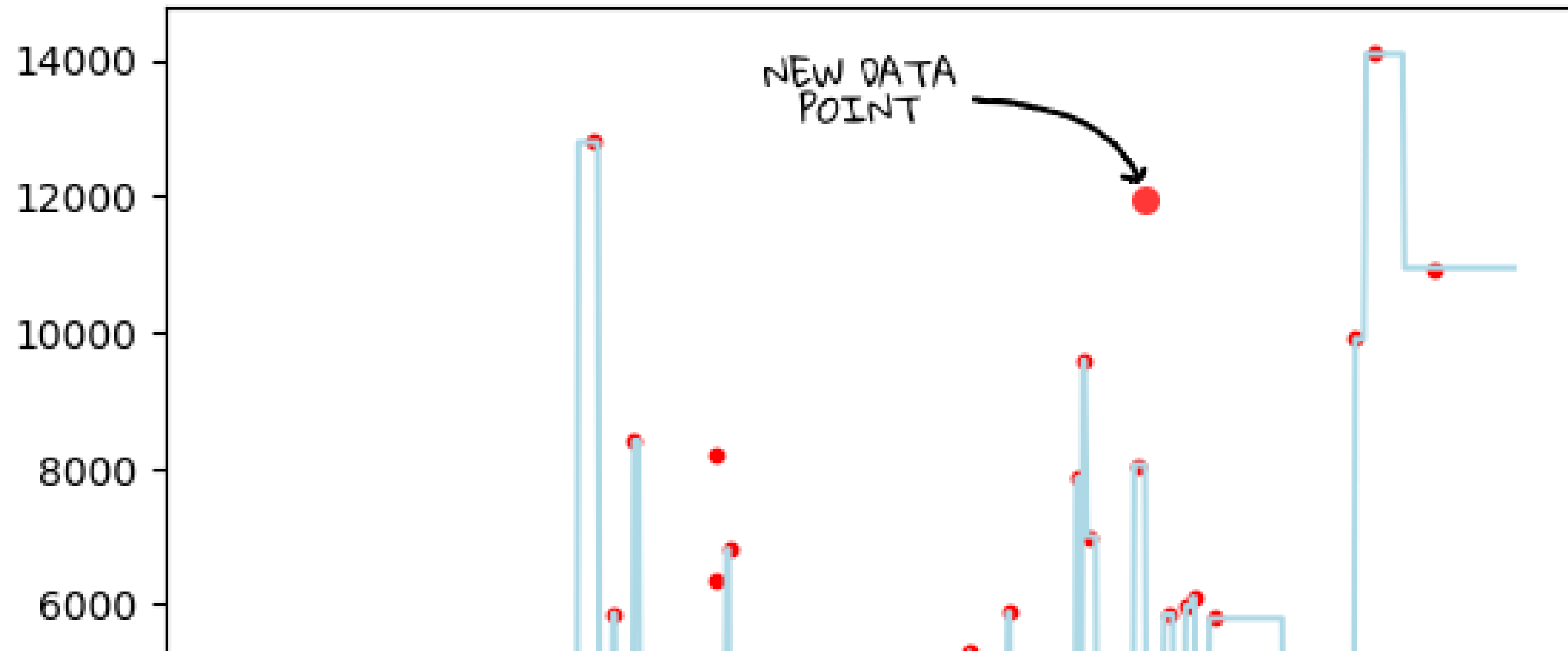
14000 -

Building the model

We now see a very complicated line that covers almost every point in our data.

What does this mean and why did it happen? It means that our model practically perfectly matched our data - usually the word "perfectly" has a very positive connotation, but not this time. In our case, it means that the model is overtrained and will not predict results well on data it has not yet seen. Let's assume that another point appears:

```
[<matplotlib.lines.Line2D at 0x7f4e0358bee0>]
```



Building the model

Predicting the value of the **y** variable for a point whose value of the **X**-variable is about 1.35 will have a very large error, due to the distance from the curve that determines the decision boundaries of the model.

We already know that the model is not good. Why is this the case? We forgot a very important thing - when creating the model, we did not define any hyperparameters. As a result, our model is maximally deep, until it fits the data perfectly. The **DecisionTreeRegressor** class allows you to set several hyperparameters that are taken into account when training the model.

Building the model

The hyperparameters of the decision tree regression model in the Scikit-learn library are:

- **max_depth** - determines the maximum depth of the tree, or in other words, the maximum number of data splits. Too large value in this parameter leads to overfitting the model, and too small - to underfitting.
- **min_samples_split** - is the minimum number of samples required to split a node into successive ones. In this case, too small value can lead to overlearning, and too large - to underlearning.
- **min_samples_leaf** - is the minimum number of samples required to form a leaf (that is, that place in the structure of the decision tree where the final result is calculated). Here, as with the minimum number of samples, too small value leads to model overfitting.

Building the model

- **max_features** - the maximum number of features that the model takes into account for training the model. To this parameter, we can pass an integer (*int*) or floating-point number (float). In the case of a floating-point number, **Scikit-learn** will round the value accordingly. In addition, there are also *sqrt* and *log₂* strategies - when passing *sqrt* to train the model, we will take the root of the number of variables, and with *log₂* - the base 2 logarithm of the number of variables.
- **criterion** - specifies the metric used to evaluate the quality of node partitioning. By default, the model uses the MSE metric, but it can also take, for example, an *absolute_error* value, corresponding to the average absolute error.
- **max_leaf_nodes** - the maximum number of *leaf nodes* in the decision tree

Building the model

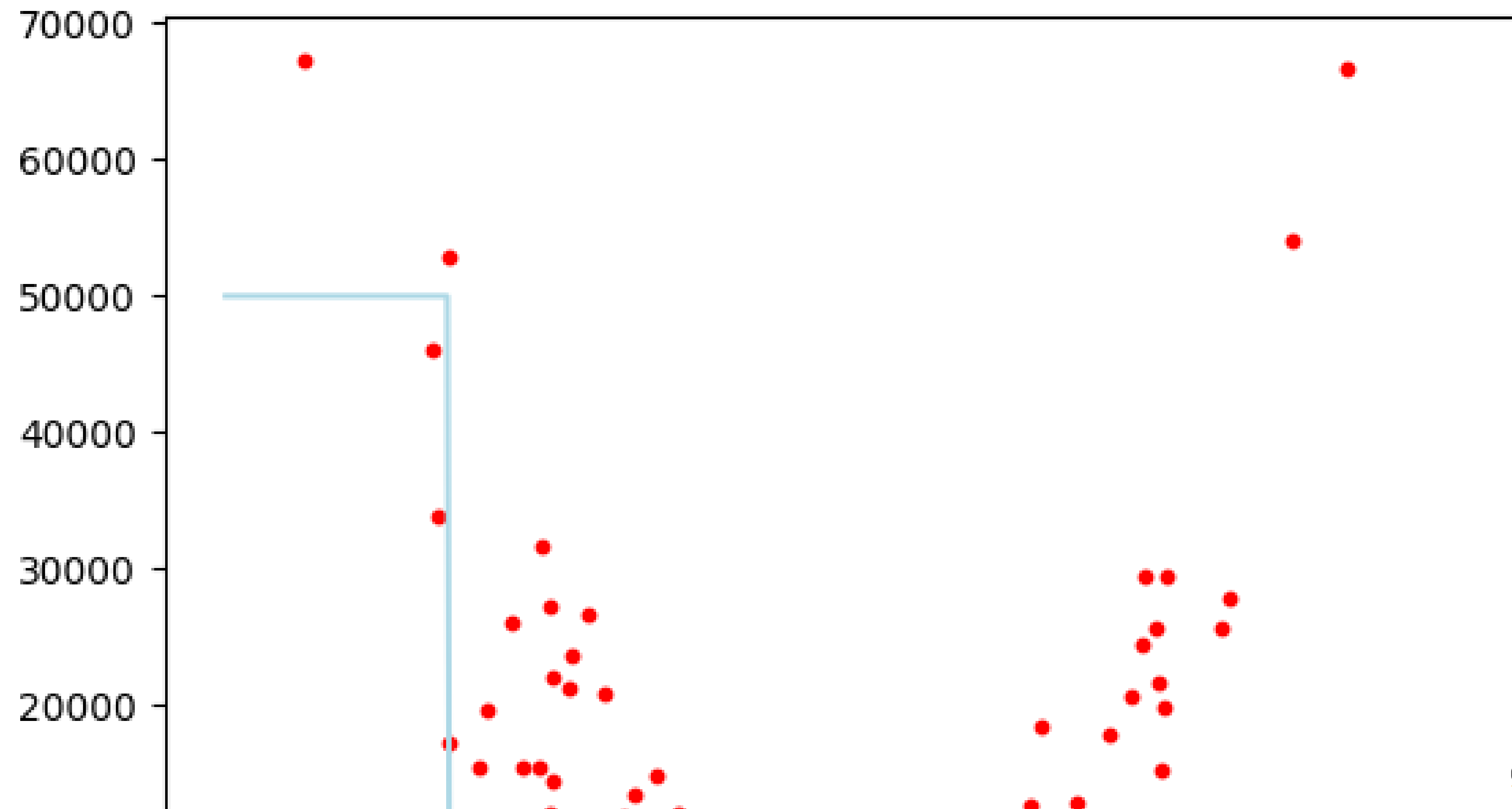
Let's return to our original model and make a change to the most relevant hyperparameter, namely **max_depth**. Let's set its value to 1, so that the decision tree will perform only one division of the data. Finally, we will again generate a graph showing the performance of the model.

```
from sklearn.tree import DecisionTreeRegressor

# Initialize decision tree model
reg_tree = DecisionTreeRegressor(max_depth=1)

# Train model
reg_tree.fit(X, y)
```

Building the model



Building the model

Scikit-learn library has a great tool that will allow us to accurately reproduce the structure of our tree, which is especially useful for datasets consisting of many variables. It is practically impossible to visualize the performance of a more complex model in the same way as for one with a single variable. We will use the *plot_tree* method from the **tree module, to which we pass a variable with the model we have trained.**

```
import matplotlib.pyplot as plt
from sklearn.tree import plot_tree

plt.figure(figsize=(12, 8))
plot_tree(reg_tree)
```

Building the model

In the generated graphic, we can see exactly how our trained tree makes decisions. The first (and only) split was made at the point where the variable **X** takes the value of -1.916, which coincides with our reading of the previous graph (there we specified that the split is made at about -2). We can see that 150 observations were taken into account in the initial split (the entire dataset we created at the beginning).

The value of the independent variable for **X** = -1.916 is 8637.19, which also coincides with the dot plot and the model's decision boundary plot. The split created two subsets of data: the first, where we have a set of points with values of **X** ≤ -1.916 and the second with values of **X** > -1.916. The first contains only 4 samples (observations), and the second contains 146. If we were to look only at our data points (without the curve showing the performance of the model), we could conclude that the distribution is unbalanced and in all likelihood - inaccurate and subject to large error. And this is indeed the case: the MSE is more than 144 billion in the data, where the **y** variable takes values in the range of 0-70000. From this it follows that the model is underfit and makes large errors.

Building the model

Let's then, check the result of the R^2 metric:

```
from sklearn.metrics import r2_score

# Make predictions and calculate R2 score
y_pred = reg_tree.predict(X)
print(r2_score(y, y_pred))
```

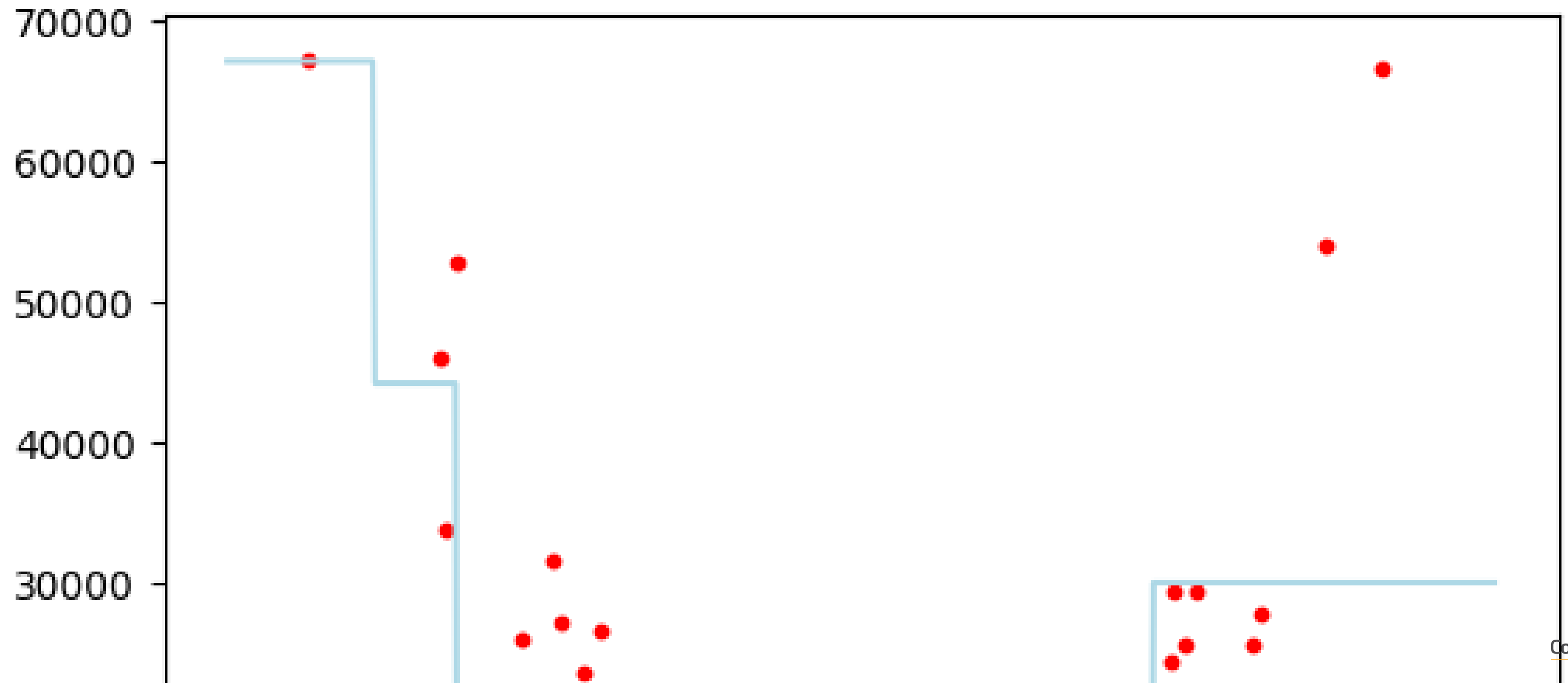
The coefficient of determination was 0.30 on the training set, so our model is severely underfit and we don't benefit from using it. Let's see how the situation changes when we increase the tree depth to 2. In our code, we only change the value of the **max_depth** hyperparameter to 2, and run the rest unchanged:

```
from sklearn.tree import DecisionTreeRegressor

# Initialize decision tree model
```

Building the model

Analyzing the graph showing how the second model works, we can see that the curve determining where data splits occur is a bit more complicated. The model captures the characteristics of the data a little better, but we can see that some of the points are still quite far from the curve.

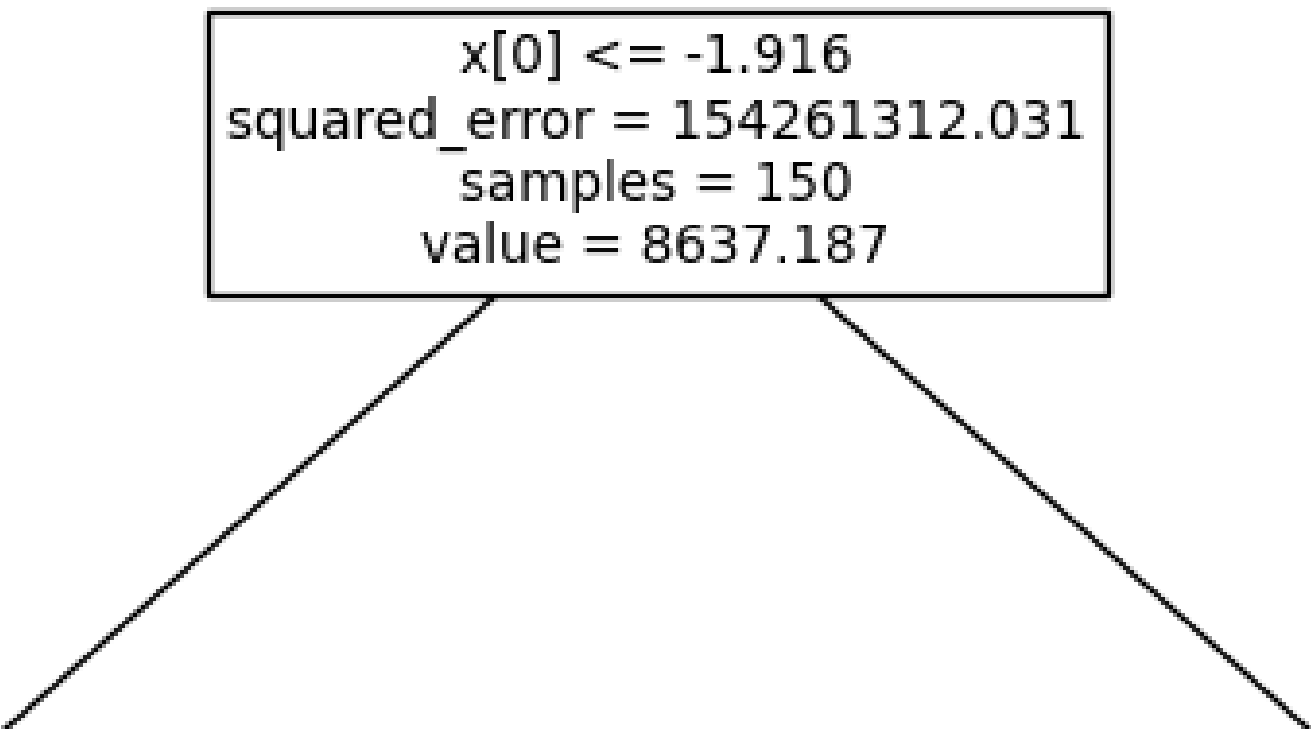


Building the model

Let's look at the tree structure then:

```
import matplotlib.pyplot as plt
from sklearn.tree import plot_tree

plt.figure(figsize=(12, 8))
plot_tree(reg_tree)
```



```
graph TD; Node["x[0] <= -1.916<br/>squared_error = 154261312.031<br/>samples = 150<br/>value = 8637.187"]
```

x[0] <= -1.916
squared_error = 154261312.031
samples = 150
value = 8637.187

Building the model

According to the hyperparameters assumed in training, we see that the model performs a maximum of two splits. Despite increasing the depth of the tree by 1, its structure has not grown. The first division is performed exactly the same.

In the branch where we have **X** values less than or equal to -1.916, the data is divided into two more subsets. One of them is **X** values less than or equal to -2.304, and the other - values greater than that. On the right side of the X axis (in the second part of our tree), the split is made when **X** = 1.38. The **value** is the average of our **y** in the given subset. Looking at the marked leaf node, we can say that among the 134 samples of this node, the average value of the sample is 5481.31.

Building the model

Let's also calculate the coefficient of determination:

```
from sklearn.metrics import r2_score

# Make predictions and calculate R2 score
y_pred = reg_tree.predict(X)
print(r2_score(y, y_pred))
```

Looking at R^2 , we can see a significant improvement in the performance of our model on the training data, which coincides with the analyzed graph and the tree structure. Let's further check how our model will behave when we increase the tree depth to 3.

Building the model

We confirmed what you probably went thought based on the knowledge you gained. A model whose depth is, 3 is a better fit to the training data, which means it is less likely to be wrong. The MSE values at the final nodes are smaller than with models with less depth.

```
from sklearn.tree import DecisionTreeRegressor

# Initialize decision tree model
reg_tree = DecisionTreeRegressor(max_depth=3)
```



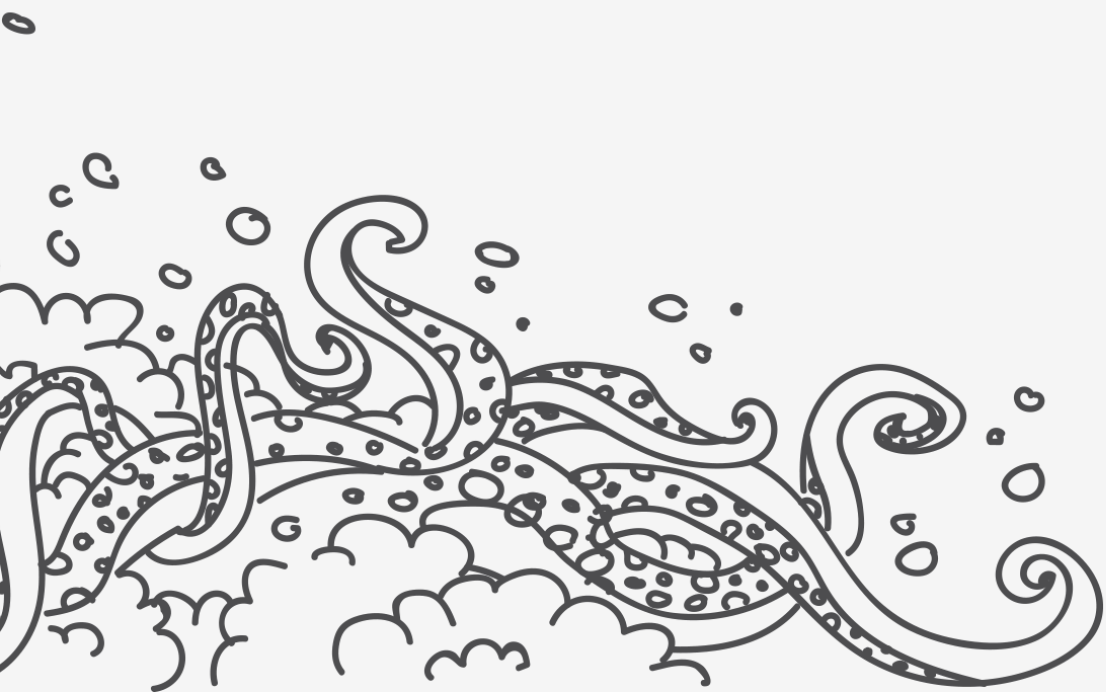
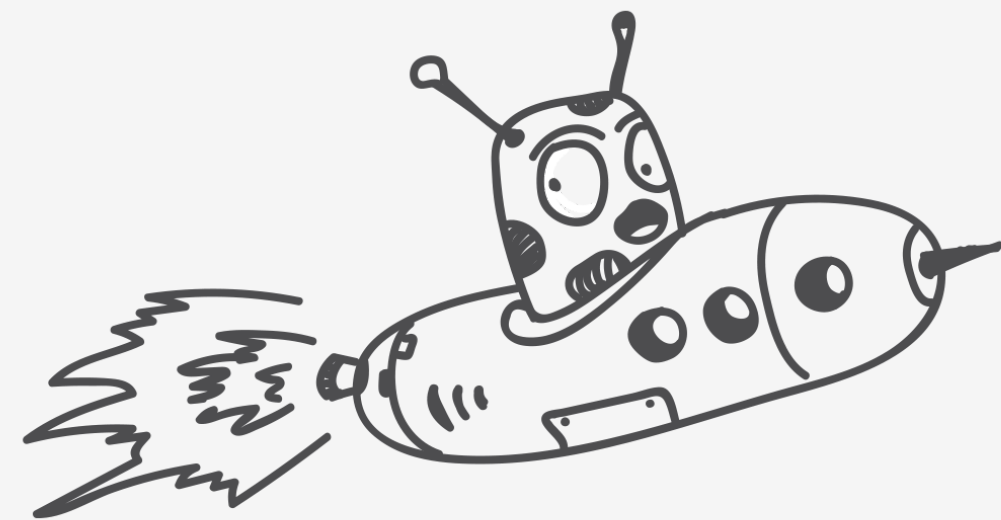
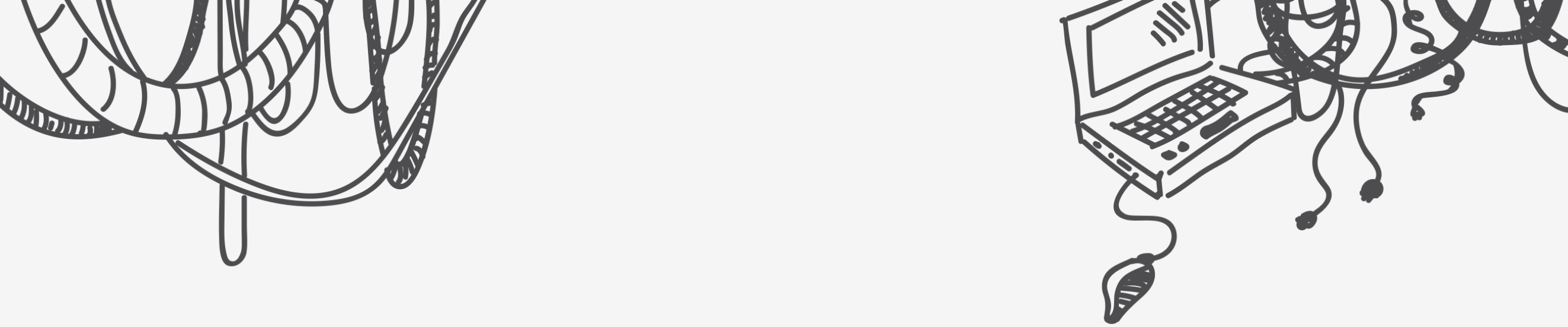
Building the model

Let's see what the R^2 metric tells us:

```
from sklearn.metrics import r2_score

# Make predictions and calculate R2 score
y_pred = reg_tree.predict(X)
print(r2_score(y, y_pred))
```

The coefficient of determination is 0.864, indicating that our model does quite well in predicting values on the set on which it was trained.

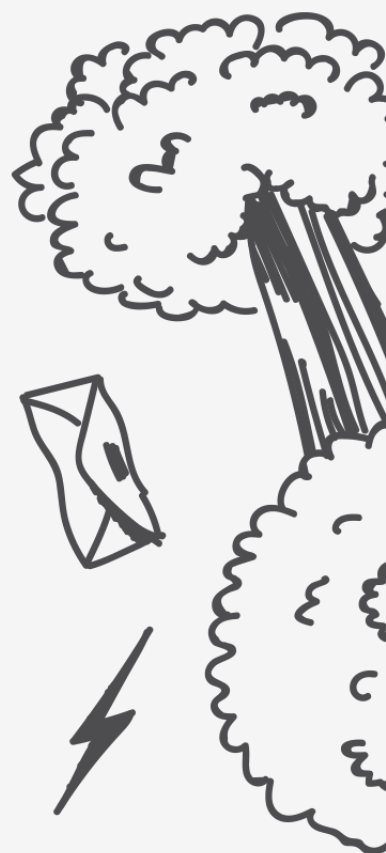
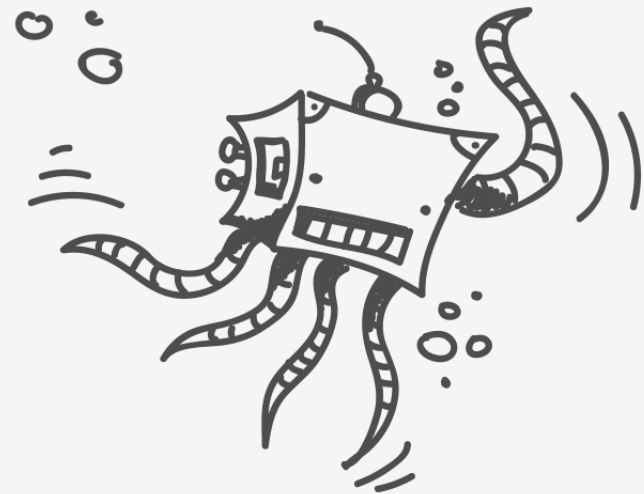
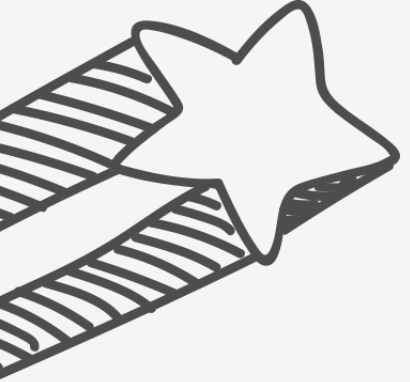


Summary

Summary

We presented tools from the **Scikit-learn** library that allow us to implement the decision tree algorithm in Python. We built a sample model and evaluated its quality on the training set. To check the actual accuracy of the model, we should determine the training set and the test set. Then the predictions on the test set would provide us with information from which we could conclude that our model is good.

The decision tree object in the Scikit-learn library has another very useful feature. The *feature_importances_* attribute stores an array that indicates how important a variable is in determining the final prediction outcome. Having this information confirms that the decision tree model is easily interpretable. We will verify the result returned by this attribute when solving a real-world problem in the **exercises**!



**Practice
time**